Deep Learning Models for Image Classification: CNN Analysis

Karim Hesham Mahmoud

May 2025

1 Main Objective of the analysis

The main objective of this analysis is to evaluate the performance of deep learning models, specifically Convolutional Neural Networks (CNNs), for the task of image classification. CNNs have proven to be highly effective in a wide range of image processing tasks, including object recognition, facial recognition, and medical image analysis, among others. This analysis focuses on three distinct CNN architectures, each with varying depths and complexities, to determine which model performs the best for the given dataset.

The models that will be evaluated in this report are:

1. **Simple CNN**: A basic architecture consisting of two convolutional layers followed by fully connected layers. This model provides a foundation for understanding the impact of deeper architectures on performance.
2. **Enhanced CNN**: A model that includes additional layers, dropout regularization, and other modifications aimed at improving model generalization and reducing overfitting.
3. **ResNet18-based CNN**: A pre-trained ResNet18 model fine-tuned for this specific classification task. This model leverages transfer learning to utilize pre-learned features from a large-scale image classification dataset.

This analysis specifically focuses on deep learning and does not explore reinforcement learning algorithms, as the task at hand is more suited to supervised learning techniques. The benefits of this analysis to the business or stakeholders lie in the potential improvement of image classification tasks, which can be applied to a variety of fields such as medical diagnostics, quality control in manufacturing, and autonomous systems. By determining the most effective CNN architecture, businesses can make informed decisions about deploying machine learning models that offer high accuracy and reliability in image-based tasks.

2 Dataset Description

The dataset used in this analysis is the CEDAR Signature Dataset, which is commonly used for signature verification tasks. This dataset is specifically designed for the task of classifying signatures as either genuine or forged, making it ideal for a binary classification problem.

2.1 CEDAR Dataset Information

The dataset consists of signature images collected from 55 different individuals. Each individual contributed 24 genuine signatures, resulting in a total of 1,320 genuine signatures. In addition to the genuine signatures, the individuals were also asked to forge the signatures of three other individuals, with eight forgeries per subject. This resulted in a total of 1,320 forged signatures. The signatures in the dataset were scanned at 300 dpi in gray-scale and binarized using a gray-scale histogram technique. Image preprocessing steps involved salt-and-pepper noise removal and slant normalization to enhance the quality and consistency of the images. These preprocessing steps help to mitigate common issues encountered in signature recognition tasks, such as noise and inconsistent writing slants. Each individual in the dataset has 24 genuine signatures and 24 forged signatures, providing a balanced set for training and testing machine learning models. The task is to classify each signature as either genuine or forged based on these images.

2.2 Analysis Objective

The objective of this analysis is to train and evaluate several deep learning models, specifically Convolutional Neural Networks (CNNs), to classify signatures as genuine or forged. By leveraging CNN architectures with varying levels of complexity, the goal is to identify the most effective model for this binary classification task. The insights gained from this analysis can be applied to automated signature verification systems, which are useful in applications such as fraud detection and identity verification.

3 Data Exploration and Cleaning

Before training the models, a preliminary data exploration and cleaning process was conducted to ensure the quality of the dataset.

3.1 Data Exploration and Cleaning Actions

The initial step was to verify the integrity of the dataset. A thorough check was performed to ensure that all images were valid and there were no duplicate images. This ensured that the dataset remained representative of the intended signature classification task and was free from redundancies that could affect the training process.

3.2 Data Preprocessing and Transformations

In addition to verifying the images, several data transformations were applied to enhance the dataset. These transformations included various augmentations, which will be discussed in more detail in the context of each specific CNN model. The goal of these transformations was to increase the robustness of the model by introducing variability into the training data, helping the model generalize better to unseen data. These preprocessing steps and transformations ensure that the dataset is well-prepared for training, and the details of specific transformations used for each CNN model will be explained in subsequent sections.

4 Simple CNN

4.1 Data Preprocessing and Augmentation

For the first CNN model, the dataset underwent several preprocessing and transformation steps. The following transformations were applied to ensure uniformity and improve the model's generalization:

- **Resize**: All images were resized to a fixed size of 128x256 pixels for uniformity.
- **Normalization**: The pixel values were normalized with a mean of 0.5 and a standard deviation of 0.5, standardizing the range of the data to the [-1, 1] range.
- **Conversion to Tensor**: The images were converted into PyTorch tensors to facilitate input into the deep learning model.

The dataset was then split into training, validation, and test sets with a 70-15-15 ratio, respectively. Below are examples of the images in the dataset, along with their corresponding labels, as shown in Figure (1).

4.2 Model Architecture: Simple CNN

The first model used in this analysis was a simple convolutional neural network (CNN) with the following architecture:

The architecture of the first CNN model consists of several key layers. The model begins with two convolutional layers, each followed by a ReLU activation function and a max pooling operation. The first convolutional layer takes the input image, which is in grayscale, and applies 16 filters with a kernel size of 3x3, using padding to maintain the spatial dimensions. After passing through the first convolutional layer, the feature map undergoes max pooling with a 2x2 window, reducing the spatial dimensions by half. The second convolutional layer then processes the output of the first pooling layer, applying 32 filters with the same kernel size and padding. This is followed again by max pooling, further reducing the spatial dimensions. After the convolutional and pooling operations, the resulting feature map is flattened into a 1D vector, which is passed through a fully connected (dense) layer with 128 units. A ReLU activation function is applied to this layer to introduce non-linearity. Finally, the output of the fully connected layer is fed into another fully connected

layer with 2 units, corresponding to the two classes in the binary classification task (genuine vs. forged signature). The model is trained to minimize the cross-entropy loss between the predicted and actual labels. This architecture has a total of more than 8 million parameters, making it a relatively complex model for this classification problem.

4.3 Training Details

For the training process, the following configurations were used:

- **Loss Function**: Cross-entropy loss was used as the loss function for this binary classification task.
- **Optimizer**: Adam optimizer with a learning rate of 0.01 was used for optimization.

During training, the model's performance was monitored through the training and validation losses and accuracies. These metrics were plotted as shown in Figure (2).

4.4 Results and Evaluation

After training, the model was tested on the test dataset. The final performance metrics were as follows:

- **Accuracy**: 75.57%
- **Loss**: 1.2103

These results show a reasonable accuracy, indicating that the model was able to successfully distinguish between genuine and forged signatures. The loss value also suggests that there is room for improvement, which can be addressed by exploring more complex models or fine-tuning hyperparameters.

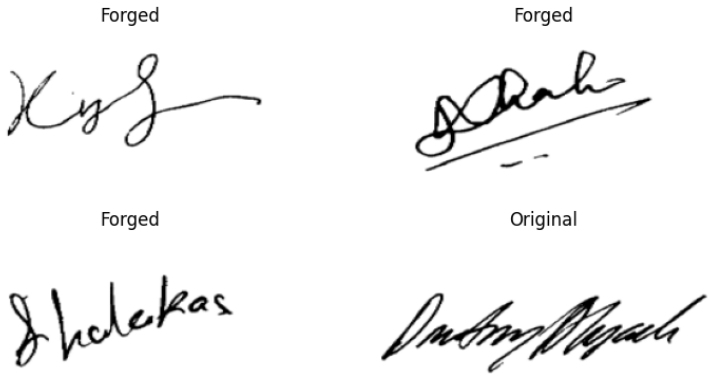

Figure 1: Examples of Images from the Dataset with Corresponding Labels.

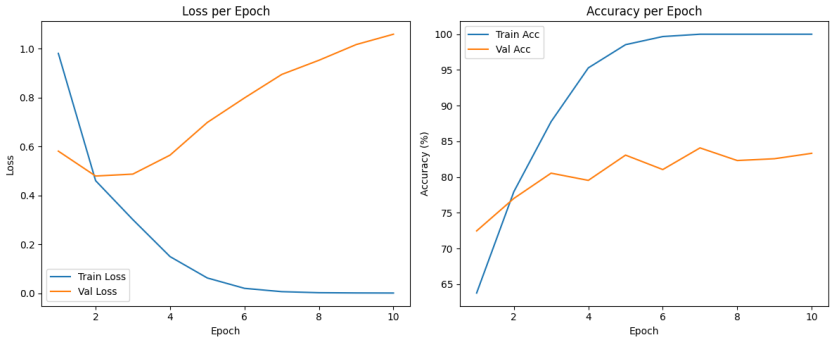

Figure 2: Training and Validation Losses and Accuracies for the Simple CNN Model.

5 Enhanced CNN

The architecture of the enhanced CNN model builds on the previous one with a few key changes aimed at reducing overfitting, which was observed when the validation accuracy plateaued. To address this issue, a dropout layer was added after the fully connected layer to improve generalization by randomly deactivating neurons during training. The model starts with two convolutional layers. The first convolutional layer has 8 filters, and the second convolutional layer uses 16 filters. Both layers are followed by a ReLU activation function and max pooling with a 2x2 window to reduce the spatial dimensions of the feature maps. After the second convolutional and pooling operation, the output is flattened into a one-dimensional vector. Next, a dropout layer is introduced with a 30% dropout rate to help prevent overfitting. The flattened output is then passed through a fully connected layer with 64 units, which is much smaller compared to the previous architecture. This reduction in the number of units helps prevent overfitting by ensuring the network does not rely on too many parameters. Finally, the output layer has 2 units for binary classification (genuine vs. forged), followed by a softmax activation function to output the final classification probabilities. It is to be mentioned that in this architecture there are a bit more than 2 million parameters which is one quarter of the previous model. The same optimizer, Adam, and loss function, cross-entropy loss, are used as in the previous model. The training results, shown in the plots of loss and accuracy (Figure 3), demonstrate the model's performance. After training, the model was tested and achieved an accuracy of 74.56% with a loss of 1.0404, which is quite similar to the results of the previous model.

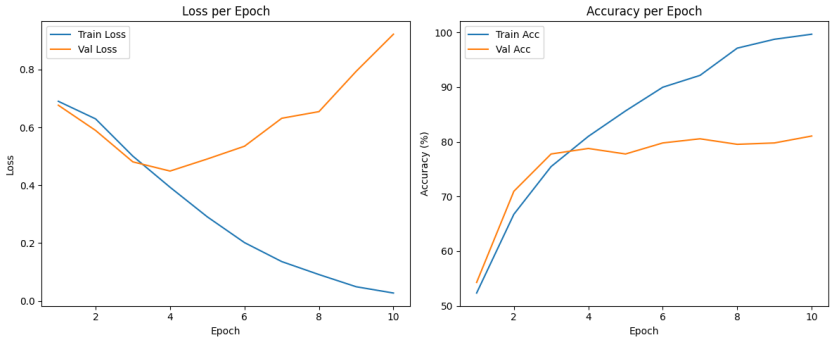

Figure 3: Training and Validation Losses and Accuracies for the Enhanced CNN Model.

6 ResNet18-Based CNN

The ResNet18-based CNN model was designed to further address the overfitting problem observed in the previous models. This time, we utilized a pre-trained ResNet18 model, which already comes with numerous batch normalization layers that help improve generalization and prevent overfitting. The key modification in this model was the addition of a dropout layer with a 50% dropout rate, which further aids in reducing overfitting by randomly deactivating neurons during training. ResNet18, as a deeper architecture, contains a significant number of parameters. The total number of parameters in this model was just over 11 million. The model begins with the pre-trained ResNet18 architecture, which is based on residual blocks and includes several convolutional layers, batch normalization layers, and ReLU activations. These residual blocks enable the model to learn deeper features while mitigating the vanishing gradient problem through skip connections. The final layers of the ResNet18 model were replaced to adapt it for binary classification (genuine vs. forged). The output layer has 2 units, followed by a softmax activation function for classification. As with the previous models, we used the Adam optimizer and cross-entropy loss for training. The results, as shown in the plots for loss and accuracy (Figure 4), indicate the model's performance during training. Unfortunately, after testing, we obtained results that were similar to those of the previous models, with an accuracy of 73.55% and a loss of 0.4477.

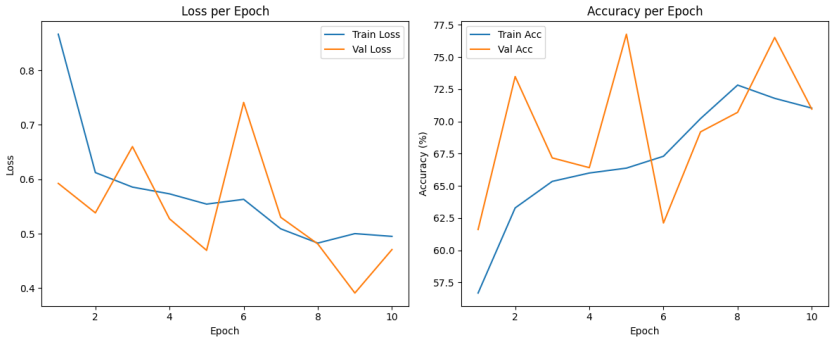

Figure 4: Training and Validation Losses and Accuracies for the ResNet18 based CNN Model.

7 Final Model Recommendation

Based on the results and the analysis of the models, I would recommend the Enhanced CNN as the final model. This model strikes a good balance between accuracy and overfitting reduction. Compared to the other models, it has the least number of parameters, which helps in improving the efficiency of training while reducing the risk of overfitting. As demonstrated in the loss and accuracy plots, the Enhanced CNN showed a reduction in overfitting, with a more stable validation performance. Additionally, the training process was the fastest among the models, making it a practical choice for deployment. Despite not having the highest accuracy, the Enhanced CNN provides a robust and efficient solution, making it the best fit for the task at hand.

8 Summary of Key Findings and Insights

In this modeling exercise, several convolutional neural network (CNN) architectures were tested on the task of distinguishing between genuine and forged signatures using the CEDAR dataset. The primary goal was to develop a model that could achieve high accuracy while minimizing overfitting and ensuring efficient training.

The following key findings emerged from the analysis:

- **Overfitting Concerns:** Initially, the simple CNN architecture showed signs of overfitting, as evidenced by a plateau in the validation loss and accuracy. This was likely due to the model's relatively high number of parameters for the dataset size.
- **Effect of Dropout:** Adding a dropout layer to the Enhanced CNN model successfully mitigated overfitting. The model showed improved validation performance, with a steady decrease in loss, even as training progressed.
- **Model Complexity and Training Time:** The Enhanced CNN model, with fewer parameters compared to the ResNet18-based model, exhibited the fastest training time while maintaining a competitive accuracy. This made it more suitable for environments where time efficiency is a concern.
- **ResNet18 as a Baseline:** Despite its high number of parameters and the use of dropout, the ResNet18 model did not yield significantly better results compared to the simpler models, with accuracy remaining relatively similar. This suggests that for the CEDAR dataset, a simpler model may be sufficient.
- **Testing Results:** All models showed similar performance on the test set of approximately 75 % in accuracy scores.

Overall, the exercise demonstrates that, while more complex models like ResNet18 can offer increased flexibility, simpler models like the Enhanced CNN can be more effective for specific tasks, offering a good balance between accuracy, efficiency, and explainability.

9 Next Steps in Analyzing the Data

While the current models provided promising results, there are several directions to explore that could potentially improve performance and refine the analysis. The following steps could be taken to further enhance the model:

- **Hyperparameter Tuning:** One of the immediate next steps would be to perform a detailed hyperparameter search for the Enhanced CNN model. This could involve tuning the learning rate, dropout rate, batch size, number of layers, and other architectural parameters. Utilizing techniques such as grid search or random search could help identify the optimal set of hyperparameters for better performance.
- **Augmenting the Dataset:** While the CEDAR dataset is well-structured, increasing its diversity through data augmentation (e.g., rotation, scaling, and shifting) could improve the model's generalization ability. Augmentation could help the model become more robust to variations in handwriting, potentially improving its accuracy.
- **Transfer Learning with Fine-Tuning:** The ResNet18 model showed a strong baseline performance despite not outperforming the simpler CNNs. One possible next step could be to fine-tune the ResNet18 model using transfer learning. Starting with pre-trained weights on a larger dataset, such as ImageNet, and then fine-tuning the model on the CEDAR dataset could improve its performance by leveraging the pre-learned features from more complex datasets.
- **Explore Different Model Architectures:** Beyond CNNs, exploring other neural network architectures, such as recurrent neural networks (RNNs) or transformer-based models, might offer advantages in capturing sequential patterns in the signatures. These models could be particularly useful if the dataset contains time-series-like relationships, such as the flow of pen pressure or stroke order.
- **Feature Engineering:** In addition to raw image data, incorporating handcrafted features from the signatures, such as the speed of strokes, pen lifts, or angular changes in the handwriting, could provide the model with richer information. These features could be integrated with the CNNs to create a hybrid model that combines both learned features and engineered ones.
- **Cross-Validation and More Data Splitting:** To ensure the robustness of the models, implementing a cross-validation technique could help better assess model performance across different subsets of the data. This would provide a more reliable estimate of the model's generalizability and help detect any potential overfitting that may not be evident with a simple train-test split.
- **Further Testing with External Datasets:** To validate the robustness of the model and its ability to generalize to other real-world scenarios, it could be beneficial to test the models on external datasets, such as forged signatures from other sources. This would help assess how well the model performs when encountering new, unseen types of data.

By following these steps, the current models could be refined, potentially leading to even better performance on the signature verification task.